

INSTITUTO POLITÉCNICO NACIONAL
UNIDAD PROFESIONAL
INTERDISCIPLINARIA DE INGENERIA y CIENCIAS SOCIALES y ADMINISTRATIVAS



Materia: Ingería de pruebas

Plan de Pruebas: Aplicación Móvil

Equipo de pruebas:

Rodríguez Mendiola Valentina (Analista)

Martínez Lagunas Andrik Jeovany (Programador)

Mejía Ramírez Luis Alejandro (Diseñador)

Jimenez Rogel Sergio (Tester)

Fecha de Elaboración: 23/11/2025

OBJETIVO

El propósito de este plan de pruebas es verificar que la aplicación móvil **BookSync 1.0** funcione correctamente y cumpla con los requerimientos funcionales, técnicos y de sincronización definidos en el proyecto.

Se busca asegurar que el sistema gestione adecuadamente el flujo completo de libros (CRUD), así como:

- Registro, edición, eliminación y visualización de libros desde SQLite.
- Sincronización correcta bidireccional entre SQLite y MySQL mediante API PHP.
- Aplicación de filtros, gestión de favoritos y personalización de colores.
- Integridad de datos ante pérdidas de conexión o fallos de red.
- Funcionamiento adecuado del menú desplegable, pantallas auxiliares y preferencias persistentes.

El plan garantizará que la aplicación proporcione:

- Correcta comunicación con el servidor mediante Retrofit/OkHttp.
- Fluidez, estabilidad y uso adecuado de corutinas.
- Persistencia y actualización adecuada de datos en ambos entornos (local y remoto).
- Experiencia intuitiva para el usuario final.

ALCANCE

El alcance incluye pruebas funcionales, de integración, sistema y seguridad sobre las siguientes áreas:

1. Gestión de libros (CRUD)

- Agregar, editar y eliminar libros localmente en SQLite.
- Subida de cambios pendientes al servidor MySQL.
- Detección de duplicados por título, autor y fecha.
- Marcado lógico de eliminaciones (*eliminado = 1*).

2. Sincronización

- Descarga inicial desde MySQL hacia SQLite.
- Comparación por fecha de modificación.
- Manejo de registros eliminados o desactualizados.

- Subida automática de cambios pendientes.

3. Interfaz de usuario (Jetpack Compose)

- Renderizado de listas, botones, formularios y pantallas completas.
- Navegación mediante menú desplegable.
- Personalización de colores mediante SharedPreferences.
- Vista de ayuda.

4. Filtrado y favoritos

- Búsqueda por título, autor y género.
- Marcar y desmarcar libros favoritos.
- Persistencia en SQLite.

5. Conectividad

- Funcionamiento sin conexión con datos locales.
- Manejo de errores de red al interactuar con la API PHP.

6. Usabilidad y estabilidad

- Fluidez general y ausencia de cierres inesperados.
- Manejo correcto de estados y corutinas.

CRITERIOS DE ACEPTACIÓN

1. Gestión de Libros

- El sistema debe permitir crear, editar y eliminar libros localmente.
- Los datos deben almacenarse correctamente en SQLite.
- La sincronización debe respetar fechas de modificación y evitar sobreescrituras innecesarias.
- Cambios marcados como *pendiente_sync = 1* deben subirse al servidor exitosamente.

2. Sincronización

- Al iniciar la app, los datos remotos deben reflejarse localmente sin errores.

- Los registros eliminados deben respetar el marcado lógico.
- No deben existir duplicados tras la sincronización.

3. Validación y formularios

- No se deben permitir campos vacíos al agregar o editar un libro.
- La validación de duplicados debe funcionar consistentemente.
- Las fechas deben estar en formato válido.

4. Favoritos y filtros

- Los favoritos deben guardarse y visualizarse correctamente.
- Los filtros deben aplicar combinaciones de criterios sin fallos.

5. Interfaz de usuario

- Los colores seleccionados deben aplicarse de manera persistente.
- Los elementos deben mostrarse correctamente con Compose.

6. Manejo de errores

- En fallos de conexión, debe mostrarse un mensaje claro.
- La app no debe cerrar inesperadamente durante sincronización.
- No debe perder información local ante fallos remotos.

RECURSOS

1. Personal de pruebas

El equipo de pruebas será responsable de ejecutar los casos definidos, registrar incidencias y validar las correcciones aplicadas antes de su entrega final.

Responsabilidades:

- Ejecutar los casos de prueba
- Registrar incidencias o defectos detectados
- Validar correcciones y confirmar el cumplimiento de los criterios de aceptación

2. Entorno de prueba

Las pruebas se ejecutarán en el siguiente entorno:

- **Sistema operativo:** Android 10, 11, 12 o superior.
- **IDE:** Android Studio.

- **Base de datos local:** SQLite.
- **Base de datos remota:** MySQL 5.7+.
- **Backend:** Servidor local Apache + PHP 7.4+.
- **Herramientas:**
    - Logcat de Android Studio
    - phpMyAdmin
    - Emuladores y dispositivos físicos
- **Tipo de entorno:** Desarrollo + pruebas en entorno local simulado.

TIPOS DE PRUEBAS

→ Pruebas de Unidad

Verificar componentes individuales:

- Inserción, edición y eliminación en SQLite.
- Validación de duplicados.
- Funciones *sincronizar()* y *subirCambiosPendientes()*.
- Filtros por título, autor y género.
- Sistema de favoritos.
- Validación de URLs con *arreglarUrl()*.

→ Pruebas de Integración

Validar la interacción entre los diferentes módulos:

- SQLite ↔ Retrofit ↔ PHP ↔ MySQL.
- Pantallas con la base de datos.
- SharedPreferences ↔ UI de personalización.
- Favoritos ↔ lista principal.

→ Pruebas de Sistema

Evaluar la app como un todo para garantizar el correcto funcionamiento bajo distintos escenarios:

- Navegación completa por toda la aplicación
- Flujo CRUD + sincronización.

- Función offline/online.
- Estabilidad general.

→ Pruebas de Aceptación del Usuario (UAT)

Validar la usabilidad del sistema con usuarios finales:

- Facilidad de uso.
- Flujo completo sin guía.
- Facilidad de agregar libros, filtrar y sincronizar.

→ Pruebas de Seguridad:

Confirmar que la aplicación maneje adecuadamente entradas inválidas y errores del servidor:

- Manejo de entradas inválidas.
- Resistencia a errores de red.
- Prevención de duplicados.
- Validación de datos antes de enviarlos al servidor.

CASOS DE PRUEBA

Registro, gestión y sincronización de libros:

1. Libro duplicado: intentar registrar un libro con mismo título, autor y fecha → debe mostrar “Libro duplicado”.
2. Campos vacíos: intentar agregar sin título o sin autor → debe solicitar llenar el campo faltante.
3. Fecha inválida: ingresar fecha en formato incorrecto → mensaje indicando que la fecha es inválida.
4. Agregar libro correcto: todos los campos válidos → registro exitoso en SQLite con *pendiente_sync = 1*.
5. Error de conexión al sincronizar: agregar libro sin conexión → guardado local con mensaje “Sincronización pendiente”.

Edición de libros:

6. Editar correctamente: modificar datos válidos → actualizar *Fecha_modificacion* en SQLite.
7. Editar con datos vacíos: borrar el título o autor y guardar → mostrar mensaje “Completa todos los campos”.
8. Editar libro duplicado: cambio provoca duplicado → debe impedirlo.

9. Sincronización tras edición: al sincronizar, cambios locales se reflejan en MySQL.
10. Conflicto de actualización (fecha menor): si edición local tiene fecha más antigua que la remota → mantener versión remota.

Eliminación de libros:

11. Eliminar un libro: marcar *eliminado = 1* en SQLite.
12. Eliminar y sincronizar: tras sincronizar, el libro debe marcarse como eliminado en MySQL.
13. Eliminar sin conexión: debe marcarse como pendiente y no borrarse del todo.
14. Reeliminación: intentar eliminar un libro ya eliminado → mostrar que ya está eliminado.

Descarga remota hacia SQLite:

15. Sincronización inicial: al abrir la app, descargar libros remotos → reflejar listado actualizado.
16. Actualizar registros modificados: si *Fecha_modificación* es mayor en MySQL → actualizar SQLite.
17. Mantener cambios locales recientes: si SQLite tiene cambios más recientes → mantenerlos (no sobrescribir).
18. Sin registros nuevos: sincronizar cuando todo está actualizado → no realizar cambios.
19. Registros eliminados en servidor: si MySQL tiene *eliminado = 1* → marcar igual en SQLite.

Subida desde SQLite a MySQL:

20. Error en servidor: fallo en PHP → mensaje de error y libros siguen como pendientes.
21. Sincronización sin conexión: debe mantener los cambios en cola.
22. Sincronización parcial: algunos libros suben y otros no → solo marcar como sincronizados los que regresen con *ok = true*.
23. Sin cambios pendientes: ejecutar sincronización → mensaje “Sin cambios por subir”.

Gestión de favoritos:

24. Marcar favorito: seleccionar un libro → debe mostrarse indicador activo.
25. Desmarcar favorito: quitarlo → eliminar marca en SQLite.
26. Persistencia: reiniciar la app → favoritos siguen guardados.
27. Filtrar solo favoritos: pantalla de favoritos muestra únicamente los seleccionados.
28. Favorito eliminado: si un libro marcado como favorito se elimina → debe desaparecer de favoritos.

Filtrado de libros:

29. Filtro por título: ingresar texto → mostrar coincidencias.
30. Filtro por autor: mismo comportamiento.
31. Filtro por género: seleccionar categoría → solo mostrar libros coincidentes.

32. Filtros combinados: título + autor → resultados correctos.
33. Filtro sin coincidencias: mostrar mensaje tipo “No se encontraron resultados”.
34. Filtro con caracteres especiales: no debe fallar la búsqueda.

Navegación:

35. Menu desplegable: abrir y navegar a cada pantalla sin errores.
36. Regresar entre pantallas: backstack correcto.
37. UI de agregar libro: campos visibles y botones funcionales.
38. Lista principal: muestra portada, título, autor y género.
39. Pantalla ayuda: visualización de texto sin errores.

Personalización de colores:

40. Cambiar color de botones: aplicarse correctamente.
41. Persistencia: reiniciar la app → conservar personalización guardada.

Conectividad:

42. Modo offline: la app debe permitir CRUD local.
43. Intento de sincronización sin internet: mostrar mensaje de error adecuado.
44. Reintentar sincronización: al volver internet, subir cambios correctamente.
45. Error 404 o 500 en API: mensaje adecuado sin cerrar la app.
46. Timeout en solicitud: mostrar “Error de conexión”.

Seguridad de datos:

47. Validación de caracteres peligrosos: probar scripts o cadenas extrañas → no debe fallar ni romper UI.
48. Inyección de SQL hacia API PHP: enviar texto malicioso → PHP debe responder con error sin comprometer la DB.
49. Manipulación de URLs: comprobar *arreglarUrl()* ante rutas inválidas.
50. Uso indebido del ID: modificar ID en editar/eliminar → debe mostrar error o ignorar solicitud.
51. Eliminaciones no autorizadas: si MySQL rechaza la solicitud → SQLite mantiene estado pendiente.

Usabilidad:

52. Flujo intuitivo agregar → editar → ver: sin confusiones.
53. Mensajes claros: error, éxito y avisos con lenguaje comprensible.
54. Iconos y botones visibles: interacción sin dificultad.
55. Comprensión sin leer la ayuda: tareas básicas pueden completarse sin instrucciones.
56. Accesibilidad mínima: textos legibles, botones accesibles.

Matriz de Casos de Pruebas Unitarias

ID	Nombre del Caso	Datos de Prueba	Acción / Procedimiento	Resultado Esperado	Tipo
CP-U001	Insertar nuevo libro	Título, autor, fecha, género	Insertar en SQLite	Registro creado con *pendiente_sync = 1*	Funcional
CP-U002	Duplicado de libro	Libro ya existente	Validar con *libroDuplicado()*	Mensaje: “Libro ya existente”	Validación
CP-U003	Campos vacíos	Falta título o autor	Validar formulario	Mensaje: “Completa todos los campos”	Límite
CP-U004	Editar libro	Datos modificados	Guardar cambios	*Fecha_modificacion* actualizada	Funcional
CP-U005	Eliminar libro	ID existente	Marcar *eliminado = 1*	Registro marcado para sincronizar	Funcional
CP-U006	Sincroni-zación local → remota	Libro con *pendiente_sync = 1*	Ejecutar *subirCambiosPendientes()*	Registro subido a MySQL	Integración
CP-U007	Sincroni-zación remota → local	Registros en MySQL	Ejecutar *sincronizar()*	SQLite actualizado correctamente	Funcional
CP-U008	Favoritos	Libro seleccionado	Marcar favorito	Persistencia en SQLite	Funcional
CP-U009	Filtros	Término de búsqueda	Aplicar filtros	Lista filtrada correctamente	Funcional

ID	Nombre del Caso	Datos de Prueba	Acción / Procedimiento	Resultado Esperado	Tipo
CP-U010	Personali-zación	Colores seleccionados	Guardar en SharedPreferences	Tema aplicado al reiniciar	Persistencia
CP-U011	Arreglo de URL	URL relativa	Ejecutar *arreglarURL()*	URL absoluta generada	Funcional

Matriz de Casos de Prueba de Integración

ID	Nombre del Caso	Precondición	Procedimiento / Pasos	Resultado Esperado	Prioridad	Tipo
CP-I001	Sincronización completa	Datos en SQLite y MySQL	1. Modificar libro local 2. Ejecutar sincronización	Datos actualizados sin conflictos	CRÍTICA	Integración
CP-I002	CRUD ↔ API PHP	Servidor activo	Crear/editar/eliminar libro	Respuesta JSON correcta	ALTA	Integración
CP-I003	SQLite ↔ Jetpack Compose	App iniciada	Insertar y ver libro en UI	Lista actualizada en tiempo real	ALTA	UI
CP-I004	Favoritos ↔ Lista principal	Libro seleccionado	Marcar y desmarcar	Vista de favoritos correcta	MEDIA	Integración
CP-I005	Personaliza-ción ↔ UI	Preferencias guardadas	Reiniciar app	Tema persistente	ALTA	Integración
CP-I006	Función offline	Sin internet	Agregar libros	Guardados con *pendiente_sync = 1*	CRÍTICA	Integración

Matriz de Casos de Prueba de Sistema

ID	Nombre del Caso	Precondición	Procedimiento / Pasos	Resultado Esperado	Prioridad	Tipo
CP-S001	Estabilidad general	App iniciada	Uso 30 minutos	Sin cierres	CRÍTICA	Estabilidad
CP-S002	Prueba de rendimiento	App iniciada	Insertar 20 libros	Respuesta < 500 ms	ALTA	Rendimiento
CP-S003	Sincronización masiva	BD remota con muchos registros	Ejecutar sincronización	Sin bloqueos	ALTA	Rendimiento
CP-S004	Fallos de red	Modo offline	Intentar sincronizar	Mensaje de error adecuado	CRÍTICA	Manejo de errores
CP-S005	Navegación completa	Sistema estable	Navegar por todas las pantallas	Funciona-miento correcto	MEDIA	Usabilidad
CP-S006	Integridad de datos	Ediciones locales	Sincronizar con remoto	Sin duplicados ni pérdidas	CRÍTICA	Datos

CRONOGRAMA

Etapa	Semana
Pruebas Unitarias	Semana 1
Pruebas de Integración	Semana 2
Pruebas de Sistema	Semana 3
Pruebas de Aceptación (UAT)	Semana 4
Pruebas de Usabilidad y Seguridad	Semana 4 (en paralelo con UAT)

INFORMES

Durante el proceso se generarán informes de estado, destacando:

- Casos de prueba ejecutados
- Defectos encontrados
- Severidad y prioridad
- Correcciones aplicadas
- Estado del avance general